From Ratings to Recommendations: Matrix Factorization on MovieLens 100k with TensorFlow.js

Ilya Bochkarev
LLM4Rec, HSE University — HW5

Site link https://bochkarev87.github.io/homework_5/
GitHub link https://github.com/bochkarev87/homework_5

Abstract — A matrix-factorization recommender with 16-dimensional user and movie factors ($\hat{r}(u,i) = \mu + p_u \cdot q_i$, clamped to [1,5]) is trained in TensorFlow.js (WebGL backend) on MovieLens 100k with a per-user chronological 80/10/10 split. In a 20-epoch run the lowest validation RMSE occurs at epoch 2, after which validation error rises while training error keeps falling. A 2-epoch re-run reaches validation RMSE 0.9983 and test RMSE 1.0414, versus 1.2289 for the training-mean baseline (about 15% lower). Recall@10 on the test split is only 0.0083.

I. Problem and Setup

The task is to predict future ratings from past ones and to produce Top-10 lists. Data: MovieLens 100k [1] (943 users, 1,682 movies, 100,000 ratings). A missing rating is unobserved data, not a zero. Each user's ratings are sorted by timestamp (ties by movieId) and split 80/10/10 by *data.js*, giving 79,619 train, 9,596 validation and 10,785 test ratings (sum = 100,000).

The model follows the standard factorization approach [2]. In *script.js* the factors are `tf.variable` matrices initialized with `tf.randomNormal(shape, 0, 0.05)`. Batches are scored with `tf.gather` and an element-wise `mul/sum`; the loss is `tf.losses.meanSquaredError` plus an L2 penalty on the factor rows used in the batch; training uses `tf.train.adam` with `optimizer.minimize` inside `tf.tidy`. Recommendation uses `matMul` and `tf.topk` with a 0/1 mask for movies rated in train or validation. Hyper-parameters: 20 epochs, learning rate 0.01, L2 = 0.05, batch size 64. The backend printed in the TensorFlow.js status panel was **webgl**.

II. Results

Self-checks. The page's "Run tests" reported 11 passed, 0 failed, 0 pending.

Training curve (validation-selected epoch in bold). Table I lists every epoch of the single 20-epoch run.

TABLE I. Training and validation RMSE per epoch

Epoch	Train RMSE	Val RMSE	Epoch	Train RMSE	Val RMSE
1	0.8870	1.0249	11	0.7021	1.0328
2	**0.7993**	**1.0072**	12	0.7003	1.0329
3	0.7571	1.0158	13	0.6996	1.0347
4	0.7324	1.0234	14	0.6999	1.0308
5	0.7240	1.0198	15	0.6985	1.0304
6	0.7158	1.0260	16	0.6988	1.0340
7	0.7119	1.0303	17	0.6988	1.0300
8	0.7085	1.0304	18	0.6973	1.0327
9	0.7070	1.0324	19	0.6977	1.0295
10	0.7048	1.0334	20	0.6966	1.0281

The minimum validation RMSE is **1.0072 at epoch 2** (train RMSE 0.7993). Afterwards training RMSE falls from 0.7993 to 0.6966 while validation RMSE rises to about 1.03, i.e. the model overfits after epoch 2 under these settings, so selecting the checkpoint by validation RMSE matters.

Test RMSE and baseline. *trainMF* does not store per-epoch weights, so after the 20-epoch run the model in memory is the epoch-20 model, not the epoch-2 one. The selected checkpoint was therefore reproduced by re-training from a fresh initialization with 2 epochs (all other hyper-parameters unchanged). Because initialization and batch order are random, this run is not identical to epoch 2 of the first run (validation RMSE

0.9983 vs. 1.0072). Its training curve was: epoch 1, train 0.8908 / val 1.0217; epoch 2, train 0.8048 / val 0.9983 (best). The test RMSE was read from the browser console with `computeRMSEFor(testSet)`, the page's

own helper.

Quantity	Value
Validation RMSE, selected epoch 2 (2-epoch re-run)	0.9983
Test RMSE of selected checkpoint	1.0414
Baseline test RMSE, computeBaselineRMSE(testSet)	1.2289
Does factorization beat the baseline?	Yes: 1.0414 < 1.2289 (difference 0.1875, about 15.3%)
Recall@10	0.0083
Excluded: unseen relevant test items	13
Excluded: users without a relevant candidate	82
Users skipped while splitting	0

Interpreting Recall@10. A Recall@10 of 0.0083 means that on average under 1% of a user's relevant test movies appear in the Top-10. As a rough back-of-envelope reference (not computed with the code), a random 10-item list drawn from the order of 1,500 eligible movies would hit about 0.6–0.7% of relevant items, so the model is only slightly above that level on this protocol. The exclusions (13 relevant test movies never seen in training; 82 users with no relevant candidate) are removed before averaging, so they do not explain the low value.

Top-10 lists. Tables II and III reproduce the page output (title, score) for the selected 2-epoch model. A score can exceed 5 (User 1, rank 1: 5.1971) because *recommendTopK* returns the raw score $\mu + p_u \cdot q_i$ used for ranking, whereas *predictRating* clamps to [1,5].

TABLE II. Top-10 for User 1

#	Title	Score
1	Wrong Trousers, The (1993)	5.1971
2	Good Will Hunting (1997)	4.9859
3	Schindler's List (1993)	4.9585
4	Casablanca (1942)	4.9475
5	Close Shave, A (1995)	4.8985
6	Bitter Moon (1992)	4.8420
7	Ruling Class, The (1972)	4.8298
8	For Whom the Bell Tolls (1943)	4.7881
9	Lawrence of Arabia (1962)	4.7587
10	Brassed Off (1996)	4.7473

TABLE III. Top-10 for User 2

#	Title	Score
1	Love & Human Remains (1993)	4.8791
2	Garden of Finzi-Contini, The (Giardino dei Finzi-Contini, Il) (1970)	4.7306
3	Wrong Trousers, The (1993)	4.6849
4	World of Apu, The (Apur Sansar) (1959)	4.6761
5	Braindead (1992)	4.6559
6	Jean de Florette (1986)	4.6546
7	Raging Bull (1980)	4.6196
8	Maya Lin: A Strong Clear Vision (1994)	4.6114
9	Lawrence of Arabia (1962)	4.5872
10	Amadeus (1984)	4.5818

Exclusion check. The console listed the movie ids rated in training or validation by each user (244 ids for User 1). The check `recommendTopK(u,10).filter(r=>s.has(r.movieId)).length` returned **0** for both User 1 and User 2, i.e. none of the twenty recommended movies was rated by its user in training or

validation.

III. Analysis: Familiar Choices vs. New Discoveries

What the lists show. User 1's list is made of widely known, well-regarded films from several decades: *Casablanca* (1942), *For Whom the Bell Tolls* (1943), *Lawrence of Arabia* (1962), *Schindler's List* (1993), *Good Will Hunting* (1997). User 2's list is more eclectic (*Braindead*, 1992; *Jean de Florette*, 1986; *Raging Bull*, 1980; *The World of Apu*, 1959). Two titles, *The Wrong Trousers* and *Lawrence of Arabia*, appear in both lists, which suggests a shared, popularity-like component in the scores in addition to user-specific taste.

Comparison with past ratings (User 1). The console listed 163 movies User 1 rated 4 or higher (from all of u.data; the output on screen was truncated after roughly 100 entries, so the following describes only the visible part). It contains many Drama titles, a block of Action/Adventure/Sci-Fi films (five *Star Trek* films, *Aliens*, *Jurassic Park*, *Terminator 2*), European art-house titles (*Three Colors: White*, *Three Colors: Blue*, *Il Postino*, *Cinema Paradiso*) and classics (*Citizen Kane*, 1941; *Psycho*, 1960). The visible history also includes *Wallace & Gromit: The Best of Aardman Animation* (1996), and the Top-10 contains two further Aardman films, *The Wrong Trousers* and *A Close Shave* — a clear repeat of an existing preference. *Casablanca* and *Lawrence of Arabia* continue the classic-cinema side of the history. The list has no Star Trek or other sci-fi/action title, so the blockbuster block of the history is not reflected. All ten movies are unrated by this user in training and validation. Genres of the ten recommended movies (from u.item): Drama in 5, Comedy in 4, War in 4, Animation in 2, Adventure in 2, Romance in 2, Thriller in 1. Drama, Comedy and War all occur in the visible part of the history, so the genres are familiar; the novelty lies in the specific titles.

Comparison with past ratings (User 2). The console listed 40 entries rated 4 or higher (*Ulee's Gold* appears twice). By genre labels, Drama occurs in 23 entries, Romance in 13, Comedy in 12, Action in 7, Crime and Thriller in 6 each. In time, 38 of the 40 entries are films from 1994–1997 (e.g. *Titanic*, *Jerry Maguire*, *Fargo*, *The English Patient*); the exceptions are *Star Wars* (1977) and *The Godfather* (1972). The Top-10 list looks different: none of its movies is from 1995–1997, nine of ten are from 1993 or earlier (*The World of Apu*, 1959; *Lawrence of Arabia*, 1962; *Raging Bull*, 1980; *Amadeus*, 1984). Genres of the Top-10: Drama in 5, Comedy in 3, and one each of Horror (*Braindead*), Documentary (*Maya Lin: A Strong Clear Vision*), Animation, Adventure, War and Mystery. Drama and Comedy repeat the main preferences, whereas Horror and Documentary do not occur anywhere in the 40 history entries, and the older films are new territory in time. This list therefore mixes a familiar genre core with some discovery.

Value for a streaming platform. Both lists suggest the model produces *plausible* candidates rather than random ones: User 1 receives more of the Aardman animations they already like (*The Wrong Trousers*, *A Close Shave*) and Drama/War classics; User 2 receives Drama and Comedy titles plus older films from outside their 1995–1997 viewing. For a platform this is useful as a way to surface back-catalogue titles that a purely recency-driven list would hide, while the Drama/Comedy core keeps the risk of a miss low. Two cautions follow from the numbers. First, *The Wrong Trousers* and *Lawrence of Arabia* appear in both lists, so part of the ranking is shared popularity rather than personalization. Second, Recall@10 is 0.0083: the test movies a user actually went on to rate highly are almost never in the Top-10, so the lists should be treated as a candidate generator for exploration and not as a final ranking; offline recall also cannot measure whether a new title would be enjoyed. These are interpretations of a single run on two users, not general findings.

IV. Verification and AI-usage Statement

The implementation of the six stubs in *script.js* was written with the AI assistant OpenCode. The assistant could not run the page (no TensorFlow.js and no dataset in its environment) and checked only JavaScript syntax. Every number above comes from the author's own runs of the page in a browser on the WebGL backend: the 11 self-checks, the 20-epoch run, the 2-epoch re-run, `computeRMSEFor(testSet)` in the console, the Top-10 lists, Recall@10, the genre tables, and the exclusion checks for both users. No hand computation of an RMSE on a single pair was performed.

References

[1] F. M. Harper and J. A. Konstan, “The MovieLens Datasets: History and Context,” *ACM Trans. Interact. Intell. Syst.*, vol. 5, no. 4, 2015, doi:10.1145/2827872.

[2] Y. Koren, R. Bell, and C. Volinsky, “Matrix Factorization Techniques for Recommender Systems,” *Computer*, vol. 42, no. 8, pp. 30–37, 2009, doi:10.1109/MC.2009.263.